CS683 PA2: Data Prefetching and Cache Replacement Summary Report

Team: Cache me if u can

Duggineni Venkata Yugesh (23B1026)
Konda Pushpa Kumar (23B0983)
Sadam Venkata Aswitha (24B1031)
Srinithya Devraj (24B0997)

1 Task 1A: the IP-stride prefetcher

`prefetcher/ip_stride.l1d_pref` keeps 24 trackers, one per load instruction, indexed by the low 8 bits of the PC and replaced with LRU. A tracker stores the last cache-line address accessed by that IP and the stride between its last two accesses. On every L1D access:

1. Look up the tracker for the IP. If none exists, allocate one (evicting the LRU tracker), record the current address with stride 0, and stop.
2. Compute `stride = current line - last line`. If it is 0 the access is to the same line and nothing is done.
3. If the new stride equals the stored stride, the IP has shown the same stride twice in a row, so the prefetcher issues `PREFETCH_DEGREE = 3` prefetches at `line + stride`, `line + 2*stride`, `line + 3*stride`. A prefetch that leaves the current 4 KB page stops the loop. Prefetches go to the L1D, or to the L2 if the L1D MSHR is more than half full.
4. Store the current address and stride in the tracker and move it to most-recently-used.

It captures only constant strides, prefetches at a fixed degree with no feedback, and with 24 trackers it loses an IP's history as soon as 24 other loads intervene. Each of these limits is addressed in Task 1B.

2 Task 1B: Stride Pattern Prefetcher (SPP)

2.1 Design

Each load IP (low 16 bits) owns one of 512 LRU trackers. The tracker holds the last access (16-bit page tag + 6-bit line offset), the last stride, a 12-bit signature of the recent stride history, and two confidence counters that put the IP in **uniform** or **non-uniform** mode.

- **Uniform mode.** Same idea as IP-stride: a stride seen twice in a row is prefetched `degree` lines ahead.
- **Non-uniform mode.** The signature is $sig = ((sig \ll 3) \oplus \text{enc}(\delta))$ & `0xFFF`, about four strides of history. A global 1024-entry pattern table maps a signature to up to four deltas that have followed it, each with a 4-bit counter, plus a `total` count of how many times the signature was seen. The confidence of a delta is $counter/total$, the fraction of times that delta actually followed this stride history. Prediction walks a path: starting at the current signature it takes the delta with the highest counter, multiplies the running path confidence by that delta's

confidence, advances the address and the signature as if the delta had happened, and repeats. The walk stops when `degree` prefetches have been produced or the path confidence drops below the threshold set by the throttle. Because confidences multiply along the path, a deep prediction built from three 60% steps counts as only 22% confident, so the prefetcher goes deep only when every step is reliable.

- **Mode switching.** Uniform to non-uniform after 2 stride mismatches; back to uniform after 4 consecutive matches. The pattern table is trained in both modes.
- **Accuracy throttling.** Every 256 prefetches that reach the lower level, accuracy = useful / lower-level prefetches is computed and sets the degree and confidence threshold:

Accuracy	Degree	Confidence threshold
$\geq 75\%$	4	0.20
50–75%	3	0.25
25–50%	2	0.35
$< 25\%$	1	0.50

  Uniform mode gets one extra degree (cap 4).
- **Lateness-driven distance.** At the same epoch boundary the prefetcher also reads the late ratio, $late/(late + useful)$, where a late prefetch is one that a demand load caught up with while it was still in flight. A high late ratio means prefetches are correct but not issued early enough. If the ratio is above 0.25 the *distance* is raised by one (cap 8); below 0.10 it is lowered by one (floor 1). In uniform mode the prefetch window is extended by $distance - 1$ lines, so the stream is prefetched further ahead of the demand access and has more time to arrive.
- Prefetches never cross the 4 KB page.

2.2 Results

All runs in this report use 25 M warm-up and 25 M simulated instructions; speedup is relative to the no-prefetcher, LRU configuration.

Trace	Prefetcher	IPC	Speedup	L1D load MPKI	Accuracy	Coverage
Trace 1	none	0.232288	1.0000	10.678	–	–
Trace 1	IP-stride	0.233344	1.0045	10.117	**53.0%**	5.3%
Trace 1	SPP	**0.234363**	**1.0089**	**9.344**	49.9%	**12.5%**
Trace 2	none	0.250638	1.0000	13.536	–	–
Trace 2	IP-stride	0.250518	0.9995	13.438	38.2%	0.7%
Trace 2	SPP	**0.255168**	**1.0181**	**12.768**	**50.8%**	**5.7%**
Trace 3	none	0.382008	1.0000	3.403	–	–
Trace 3	IP-stride	0.382889	1.0023	3.319	16.0%	2.5%
Trace 3	SPP	**0.391716**	**1.0254**	**2.734**	**40.1%**	**19.7%**

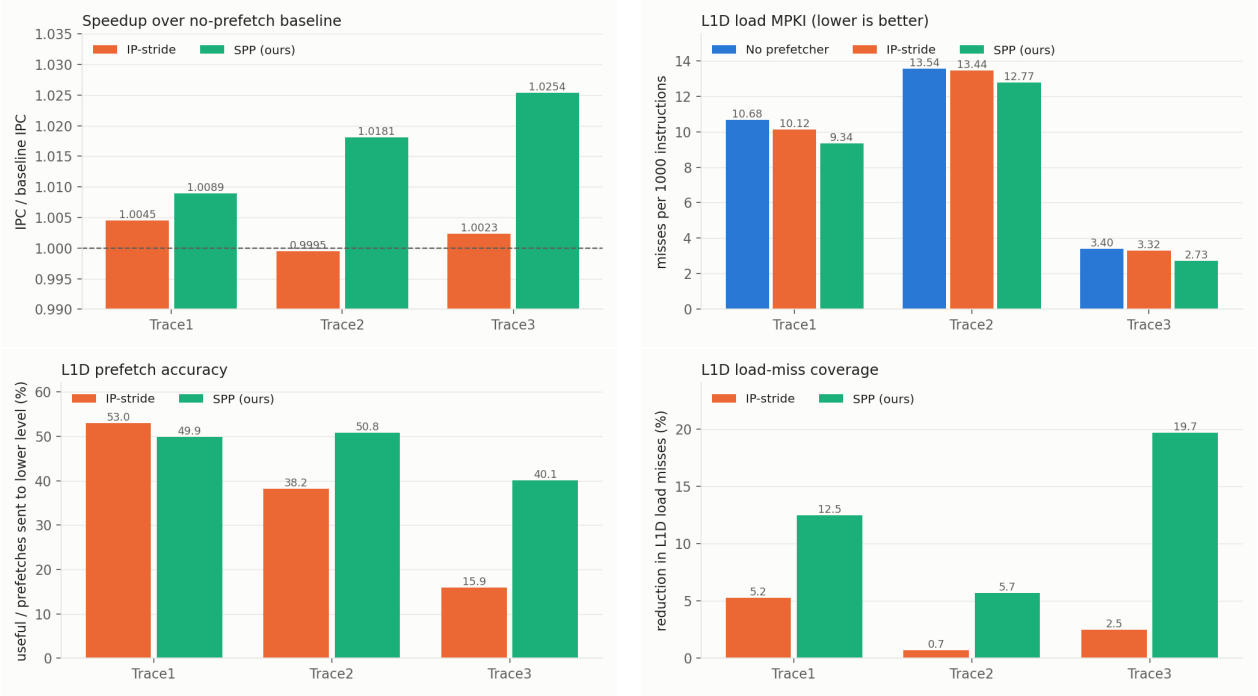


Figure 1: Task 1B: speedup, L1D load MPKI, prefetch accuracy and coverage.

SPP removes 2 to 8 times as many L1D misses as IP-stride and is more accurate on two of three traces. Speedups are small in absolute terms because all three traces are front-end bound: the processor spends most of its time waiting on instruction fetch and branch mispredictions rather than on data, so removing data misses moves IPC by only a few percent.

2.3 Storage overhead

Structure	Per-entry fields	Bits
Pattern table, 1024 entries	4 deltas × 7, 4 counters × 4, 4 valid, total 8	$1024 \times 56 = 57\,344$
IP trackers, 512 entries	IP tag 16, LRU 9, mode 1, valid 1, page tag 16, line offset 6, prefetch pointer 1+6, last stride 7, signature 12, two confidences 4+4	$512 \times 83 = 42\,496$
Throttle state	degree 2, threshold selector 2, distance 3, three 9-bit epoch counters	34
Total		99 874 bits = **12.2 KB**

2.4 Throttling threshold and degree choices

Accuracy is measured against prefetches that reach the lower level, not all issued prefetches, because most issued prefetches hit lines already in the L1D or in flight and can never count as useful. Accuracy is re-evaluated every 256 such prefetches. A prefetch takes some time before a demand access can hit it, so a window shorter than this would often count prefetches as not useful before they had a chance to be; a much longer window would be slow to notice when the program's behaviour changes.

The 25/50/75% bands follow the high/low accuracy boundaries of Feedback Directed Prefetching, split into four equal bands so each maps to one degree step. All three traces visit all four bands and have different modal bands, which shows the throttle is doing work.

Degrees 1 to 4. The L1D prefetch queue and MSHR hold 16 entries each, so degree 4 is the most one access should generate; ChampSim reports no dropped prefetches at this cap. Degree 0 is not used because no prefetches would mean no accuracy sample and no way to recover. One step per band was preferred over a steeper schedule because the traces spend most epochs in the two middle bands. The confidence threshold rises as the degree falls so that in the low band the non-uniform path becomes stricter as well as shorter. The most important sizing decision was the

tracker count: going from 24 to 512 trackers lifted Trace 2 coverage from 0.8% to 5.7%, and trackers are worth far more per kilobyte than pattern-table entries.

3 Task 2A: Instruction-Conscious RRIP (I-RRIP)

3.1 Design

The replacement hook was extended with an `isInstruction` parameter, set from the packet's `instruction` flag. Each L2 line keeps a 2-bit RRPV and a 1-bit instruction flag. One line was also added to `src/main.cc` so that `l2c_replacement_final_stats()` is called at the end of the run, since the provided tree never printed replacement statistics.

- **Insertion (changed).** Instruction lines enter at RRPV 1, data lines at RRPV 2 as in SRRIP. An instruction line therefore survives one more ageing round than a data line before becoming a candidate.
- **Promotion (unchanged).** Any hit sets RRPV 0. A hit is evidence of reuse for both line types, and promoting instruction lines further than data lines is not possible below 0.
- **Eviction (changed).** The victim is a line at RRPV 3 that is not a protected instruction line; if none, age the set and repeat. A protected instruction line is skipped by the victim search even when its RRPV has reached 3, so a data line at RRPV 3 is evicted instead. Instruction lines are protected only while the set holds at most 4 of them (and at least one data line).

The cap of 4 protected ways is the balance point. Protecting more ways lowers instruction MPKI further but raises data MPKI faster:

Protected ways	Speedup T1 / T2 / T3	L2 instr MPKI T1 / T2 / T3	L2 data MPKI T1 / T2 / T3
2	1.0001 / 1.0095 / 1.0007	10.22 / 17.21 / 0.98	7.43 / 8.53 / 2.90
4 (chosen)	0.9991 / 1.0135 / 1.0032	9.92 / 16.09 / 0.86	7.94 / 8.91 / 2.99
6	1.0046 / 1.0305 / 1.0056	9.36 / 11.35 / 0.77	9.15 / 10.87 / 3.15
8	1.0143 / 1.0386 / 1.0065	9.06 / 9.45 / 0.73	10.02 / 12.40 / 3.27

Unlimited protection gives the highest IPC on these traces, but it raises data misses by 36 to 43% and would behave badly on any data-heavy phase. Four ways keeps the data penalty small (8% on Trace 1) while still cutting instruction misses on every trace.

3.2 Results

Trace	Policy	IPC	Speedup	L2C load MPKI	L2 instr MPKI	L2 data MPKI
Trace 1	LRU	0.23229	1.0000	17.751	10.403	7.348
Trace 1	RRIP	**0.23232**	**1.0001**	**17.681**	10.613	**7.068**
Trace 1	I-RRIP	0.23208	0.9991	17.854	**9.917**	7.938
Trace 2	LRU	0.25064	1.0000	28.229	19.530	8.699
Trace 2	RRIP	0.25130	1.0026	26.607	18.546	**8.061**
Trace 2	I-RRIP	**0.25403**	**1.0135**	**25.001**	**16.091**	8.910
Trace 3	LRU	0.38201	1.0000	3.959	1.123	**2.836**
Trace 3	RRIP	0.38163	0.9990	4.044	1.198	2.847
Trace 3	I-RRIP	**0.38323**	**1.0032**	**3.846**	**0.857**	2.989

I-RRIP reduces L2 instruction MPKI on every trace (5%, 18% and 24%). It wins on Trace 2 and Trace 3 and loses slightly on Trace 1, where the data-miss increase outweighs the instruction gain. Plots for all Task 2 policies are in Figure 2.

4 Task 2B: Dynamic I-RRIP

4.1 Design

The 1024 L2 sets are split into 32 regions of 32 sets; one set per region (sets 0, 33, 66, ...) is a **leader** and always runs plain SRRIP. The other 992 sets are **followers**. Every demand access (load or RFO) to a leader set updates four counters: instruction accesses, instruction misses, total accesses, total misses. After 4096 leader accesses an **epoch** closes and the followers' policy for the next epoch is chosen by

$$R = \frac{\text{instr hits}}{\text{instr misses} + 1} \times \frac{\text{instr accesses}}{\text{total accesses}}, \qquad \text{followers run I-RRIP if } R > 0.15, \text{ else SRRIP.}$$

The counters are then zeroed, so each decision reflects the last epoch only. I-RRIP mode is the Task 2A policy with one extra eviction rule: a per-line *reused* bit (set on hit, cleared on fill) makes never-reused lines the first victims among the RRPV-3 candidates. Storage beyond SRRIP: instruction flag and reused flag per line (2 KB total), four 13-bit counters and a mode bit.

4.2 Results

Trace	Policy	IPC	Speedup	L2C load MPKI	L2 instr MPKI	L2 data MPKI
Trace 1	I-RRIP (2A)	0.23208	0.9991	17.854	**9.917**	7.938
Trace 1	Dynamic I-RRIP	**0.23228**	**1.0000**	**17.812**	10.396	**7.416**
Trace 2	I-RRIP (2A)	0.25403	1.0135	25.001	16.091	8.910
Trace 2	Dynamic I-RRIP	**0.25457**	**1.0157**	**24.479**	**15.621**	**8.857**
Trace 3	I-RRIP (2A)	**0.38323**	**1.0032**	**3.846**	**0.857**	2.989
Trace 3	Dynamic I-RRIP	0.38322	**1.0032**	3.849	0.862	**2.987**

Geomean speedup: LRU 1.0000, RRIP 1.0006, I-RRIP 1.0052, Dynamic I-RRIP 1.0063. On Trace 1 the controller keeps the followers in SRRIP for 7 of 10 epochs and removes the Task 2A loss. On Trace 2 it stays in I-RRIP for 16 of 17 epochs and the reused bit lowers instruction misses a further 3%. Trace 3 stays in I-RRIP for 4 of 5 epochs and matches 2A. The per-trace epoch counts reported by the policy are:

Trace	Epochs	In I-RRIP mode	In SRRIP mode	Mode switches
Trace 1	10	3	7	3
Trace 2	17	16	1	3
Trace 3	5	4	1	3

4.3 Sensitivity study

Threshold sweep on R (32 leaders, 4096-access epochs), speedup over LRU, with the number of epochs spent in I-RRIP:

Threshold	Trace 1	Trace 2	Trace 3	Geomean	I-RRIP epochs T1 / T2 / T3
0.05	0.9995	1.0157	1.0032	1.0061	10/10, 16/17, 5/5
0.10	0.9989	1.0157	1.0032	1.0059	4/10, 16/17, 5/5
0.15	**1.0000**	**1.0157**	**1.0032**	**1.0063**	3/10, 16/17, 4/5
0.20	1.0000	1.0144	1.0003	1.0049	3/10, 14/17, 2/5
0.25	1.0001	1.0144	1.0003	1.0049	2/10, 14/17, 2/5

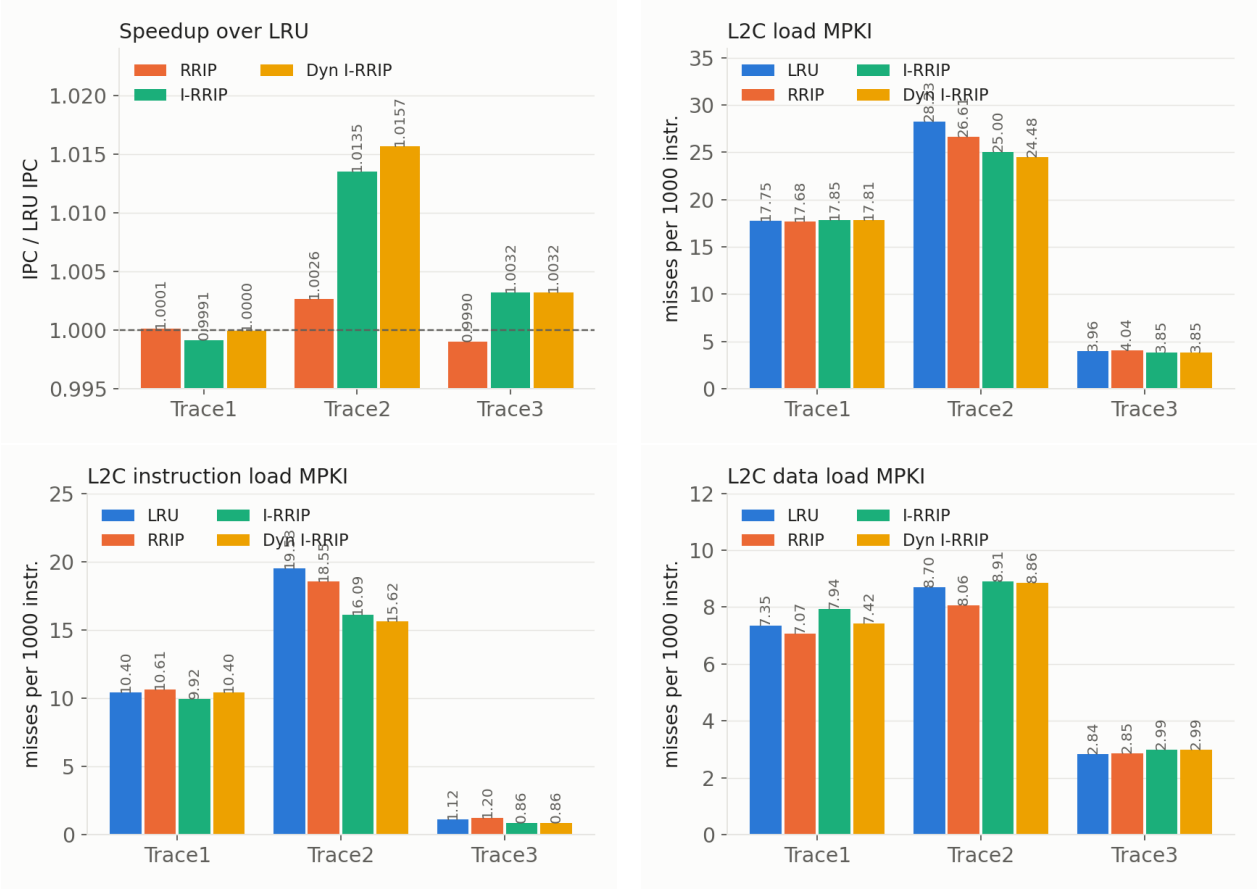


Figure 2: Tasks 2A and 2B: speedup and L2 load MPKI (total, instruction, data) for all policies.

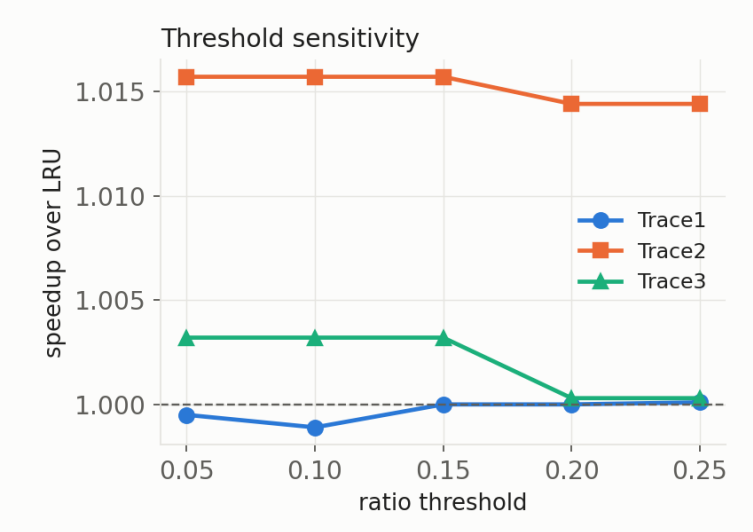


Figure 3: Task 2B: threshold sensitivity.

Leader-set sweep at threshold 0.15: 16 leaders 1.0063, 32 leaders 1.0063, 64 leaders 1.0047 geomean.

4.4 Rationale

Metric. The hinted ratio, (instr accesses + misses) / (total accesses + misses), only measures how much instruction traffic there is. Trace 1 and Trace 3 both score about 0.46 on it yet want opposite policies, so no threshold works for both. Our ratio R multiplies instruction intensity by instruction reuse (hits per miss), which is what decides whether protecting code pays off. Under R, Trace 1's epochs sit below 0.3, Trace 2's above 0.7 and Trace 3's are bimodal; 0.15 is the one value that separates them correctly, as the sweep confirms.

Leader sets. 32 leaders, one per 32-set region, is the usual dueling ratio for a 1024-set cache. 16 performs the same but makes very few decisions per run; 64 fixes too many sets to SRRIP, and because each epoch then needs fewer accesses per set, the decision becomes less reliable on Trace 3. Leaders are spread evenly across the cache so that they see a fair sample of all accesses.

Epoch. 4096 leader accesses. Shorter epochs saw Trace 3's bursty instruction reuse as mostly misses and dropped into SRRIP during the phases where I-RRIP helps; longer epochs made the policy effectively static.

5 Task 3: SPP with prefetch-aware Dynamic I-RRIP

5.1 Design

Every SPP prefetch that misses the L1D reaches the L2 as a `PREFETCH` packet. Two rules were added to the Task 2B policy; everything else, including the controller, is unchanged.

1. **Prefetched lines insert at RRPV 3** in both modes, so they are the first eviction candidates unless a demand access touches them first.
2. **A prefetch hit does not promote** the line or set its reused bit. Only demand hits count as reuse.

Both rules follow from a measurement in the leader sets: only about 1% of prefetched L2 lines are ever demand-hit at the L2, even though SPP itself is about 50% accurate. A useful SPP prefetch is consumed in the L1D, so its L2 copy is dead. Inserting such a line at RRPV 2 would spend ageing rounds on it and push protected instruction lines towards eviction for no benefit. Ignoring prefetch hits keeps the reused bit meaningful for the never-reused-first eviction rule. The ratio's counters already count only loads and RFOs, so prefetch packets never influence the mode decision. No extra storage is needed.

5.2 Results

Configurations: *2B alone* is Dynamic I-RRIP with no prefetcher; *SPP + 2B* is SPP with the unmodified Task 2B policy; *SPP + 3* is SPP with the prefetch-aware policy.

Trace	Config	IPC	Speedup	Accuracy	Coverage	L1D MPKI	L2C MPKI	L2 instr MPKI	L2 data MPKI
Trace 1	2B alone	0.23228	1.0000	–	–	10.678	17.812	10.396	7.416
Trace 1	SPP + LRU	0.23436	1.0089	**49.9%**	**12.5%**	9.344	16.706	10.436	**6.270**
Trace 1	SPP + 2B	0.23441	1.0091	**49.9%**	12.4%	9.351	16.629	10.286	6.343
Trace 1	**SPP + 3**	**0.23451**	**1.0095**	**49.9%**	**12.5%**	**9.341**	**16.503**	**10.217**	6.286
Trace 2	2B alone	0.25457	1.0157	–	–	13.540	24.479	**15.621**	8.857
Trace 2	SPP + LRU	0.25517	1.0181	**50.8%**	**5.7%**	12.768	27.755	19.852	**7.903**
Trace 2	SPP + 2B	0.25923	1.0343	50.2%	**5.7%**	**12.766**	23.932	15.864	8.069
Trace 2	**SPP + 3**	**0.25932**	**1.0346**	50.7%	**5.7%**	12.768	**23.796**	15.749	8.047
Trace 3	2B alone	0.38322	1.0032	–	–	3.403	3.849	**0.862**	2.987
Trace 3	SPP + LRU	0.39172	1.0254	**40.1%**	**19.7%**	**2.734**	3.125	1.147	**1.978**
Trace 3	SPP + 2B	**0.39311**	**1.0291**	39.8%	19.4%	2.744	2.988	0.864	2.124
Trace 3	**SPP + 3**	0.39301	1.0288	39.8%	19.4%	2.742	**2.983**	0.863	2.121

Geomean speedup: SPP alone 1.0174, Dynamic I-RRIP alone 1.0063, SPP + Dynamic I-RRIP 1.0241, SPP + prefetch-aware Dynamic I-RRIP **1.0243**. The combined policy beats both standalone SPP and standalone Dynamic I-RRIP on every trace. SPP removes L1D data misses and Dynamic I-RRIP removes L2 instruction misses, so their gains add. The prefetch-aware rules lower L2C load MPKI on all three traces and raise IPC on Trace 1 and Trace 2; Trace 3 is within noise. The effect is small because prefetches are only 3 to 28% of L2 traffic.

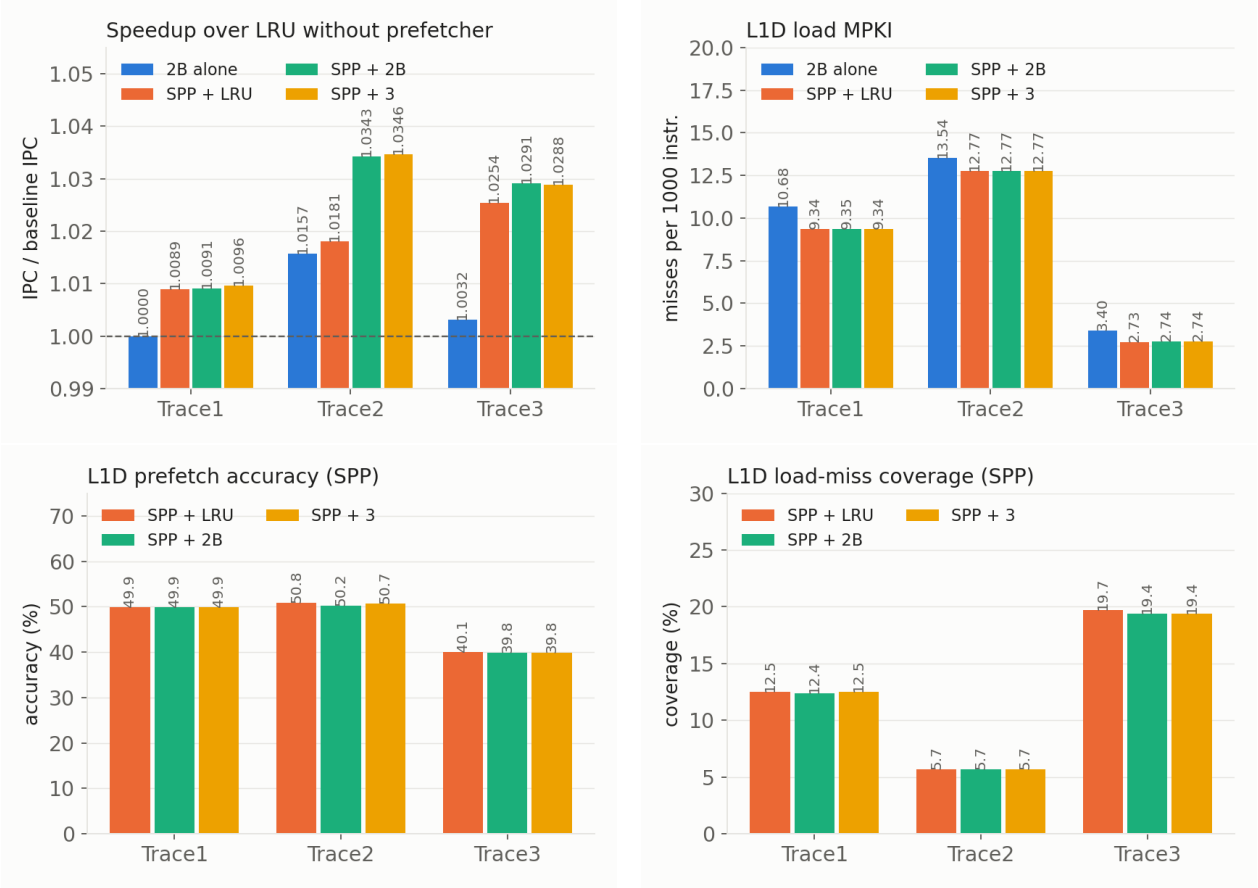


Figure 4: Task 3: speedup, L1D load MPKI, prefetch accuracy and coverage.

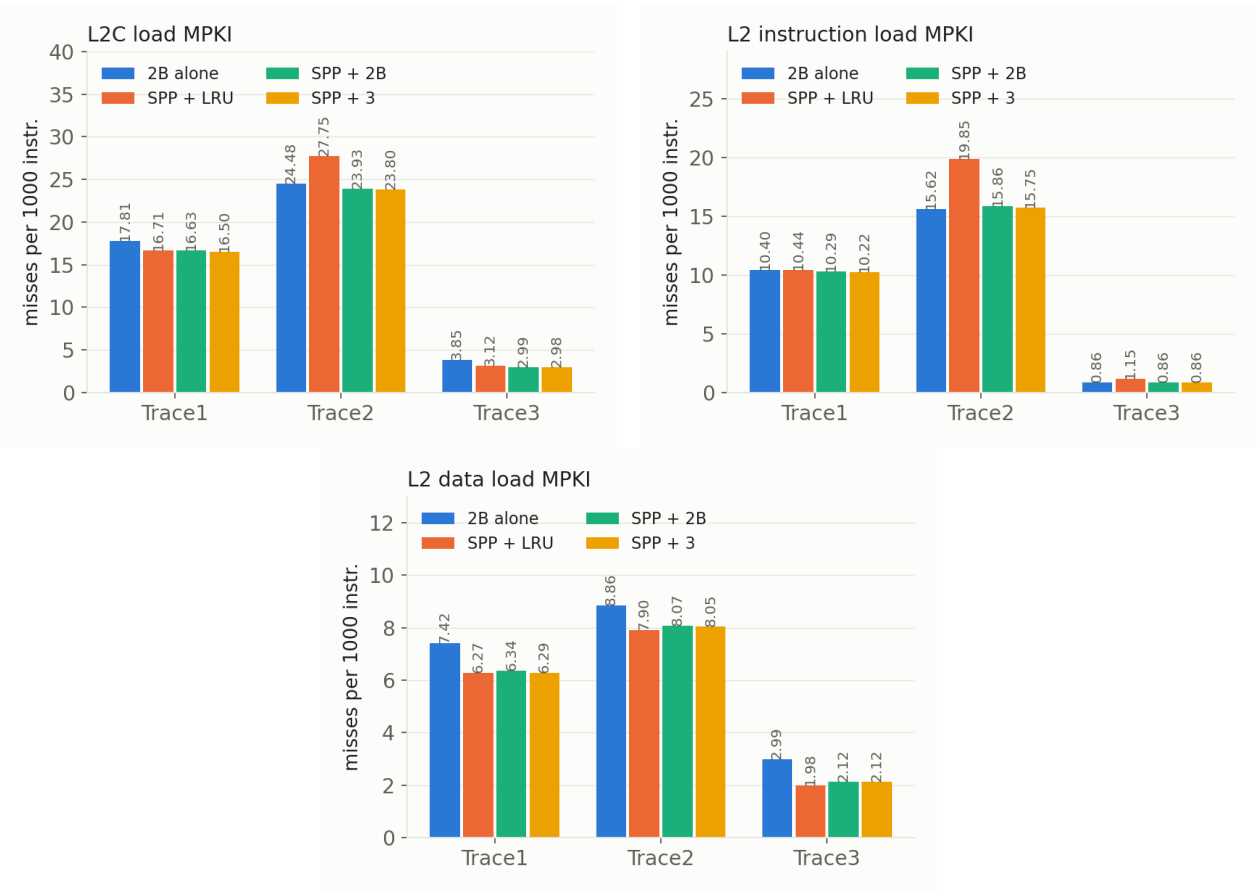


Figure 5: Task 3: L2C, L2 instruction and L2 data load MPKI.